

A BUILD-BY-DOING TUTORIAL

FastAPI Zero to Production

From “what is an API?” to a deployed, tested, authenticated backend — one phase at a time.

8

PHASE EIGHT

Deployment with Docker

Ship your API like a professional: containerise with a Dockerfile, run app plus PostgreSQL via docker-compose, configure production server settings and environment variables, and deploy live.

Contents

8.1 Why Docker?	4
The "works on my machine" problem	4
Container vs virtual machine	5
The four Docker concepts	5
8.2 Writing the Dockerfile	5
Multi-stage builds (keeping images small)	6
The .dockerignore file	7
8.3 docker-compose.yml	7
8.4 Running in Production Mode	9
Multiple workers	9
Gunicorn + Uvicorn workers (the recommended setup)	9
Running migrations on container startup	10
8.5 Environment Variables for Production	10
8.6 Deployment Platforms Overview	11
Step-by-step: deploy free on Render	11
8.7 Reverse Proxy with Nginx (Concept)	12
Hands-On Exercise 8: Dockerise and deploy the Book Library	12
Checkpoint: At This Point You Can Now...	14
Phase 8 Summary	15

Your Journey

✓	Foundations: What Is an API?	DONE
✓	Setup & Your First API	DONE
✓	Path Params, Query Params & Body	DONE
✓	Pydantic: Data Validation	DONE
✓	Database Integration (SQLAlchemy)	DONE
✓	Authentication & Authorization	DONE
✓	Advanced FastAPI Features	DONE
✓	Testing Your FastAPI App	DONE
8	Deployment with Docker	YOU ARE HERE
★	Capstone — TaskFlow API	

Your API works, it's secure, and it's tested. But it only runs on *your* machine. Deployment is the step that makes it available to the world — and historically it's where projects get stuck, because "it works on my laptop" rarely means "it works on the server." This phase fixes that with Docker, the tool that packages your app and everything it needs into a portable container that runs the same everywhere. By the end you'll have a live URL anyone can visit.

8.1 Why Docker?

The "works on my machine" problem

Your app depends on a specific Python version, specific package versions, system libraries, and environment variables. The server has different versions of all of these. So code that runs perfectly for you fails mysteriously in production. This mismatch has wasted countless developer-hours.

Docker solves it by packaging your app *together with* its exact environment — the right Python, the right packages, the right OS libraries — into a single unit called a **container**. That container runs identically on your laptop, your teammate's laptop, and the production server. "Works on my machine" becomes "works on every machine."

Container vs virtual machine

A **virtual machine** virtualises an entire computer, including a full operating system — heavy (gigabytes) and slow to start. A **container** shares the host's OS kernel and packages only your app and its dependencies — lightweight (megabytes) and starts in seconds. The analogy: a VM is like moving house (take the whole building); a container is like a shipping container (standardised box that any ship, truck, or crane can handle without knowing what's inside).

The four Docker concepts

Term	What it is	Analogy
Image	A read-only template: your app + environment, built once	A recipe / class
Container	A running instance of an image	A cooked meal / object
Dockerfile	The text file of instructions to build an image	The recipe's written steps
docker-compose	A tool to run multiple containers together	A meal plan coordinating several dishes

CONCEPT CHECK

Concept Check: What's the difference between an image and a container? *Answer: An image is the static template (built from your Dockerfile); a container is a live, running instance of that image. One image can spawn many identical containers — like one class producing many objects.*

8.2 Writing the Dockerfile

A **Dockerfile** is a list of steps Docker follows to build your image. Here's a solid one for a FastAPI app, line by line:

DOCKERFILE

```
# Start from a small official Python image (slim = fewer extras, smaller size).
FROM python:3.11-slim

# Set the working directory inside the container.
WORKDIR /app

# Copy ONLY requirements first, so Docker can cache the pip install layer.
COPY requirements.txt .

# Install dependencies. --no-cache-dir keeps the image smaller.
RUN pip install --no-cache-dir -r requirements.txt

# Now copy the rest of the application code.
COPY . .

# Document which port the container listens on.
EXPOSE 8000

# The command that runs when the container starts.
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Two details that matter:

- **Copy `requirements.txt` before the code.** Docker builds in cached layers. If you copy everything at once, *any* code change busts the cache and reinstalls all packages every build. By copying requirements first, the slow `pip install` layer is reused unless your dependencies actually change — builds go from minutes to seconds.
- **`--host 0.0.0.0`** (not `127.0.0.1`). Inside a container, `127.0.0.1` means "only reachable from inside this container." `0.0.0.0` means "reachable from outside," which is what you need.

Multi-stage builds (keeping images small)

For leaner production images, a **multi-stage build** compiles/installs in one stage and copies only what's needed into a clean final stage, leaving build tools behind:

DOCKERFILE

```
# ---- build stage: install into a virtual environment ----
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN python -m venv /venv && /venv/bin/pip install --no-cache-dir -r requirements.txt

# ---- final stage: copy just the venv + code, no build tools ----
FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /venv /venv
COPY . .
ENV PATH="/venv/bin:$PATH"
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

The `.dockerignore` file

Like `.gitignore`, it keeps junk out of your image — smaller, faster, safer builds:

TEXT

```
venv/
__pycache__/
*.pyc
.env
.git
tests/
*.db
```

COMMON MISTAKES

Common Mistakes: Forgetting `.dockerignore` and copying your local `venv/`, `.git/`, or `.env` into the image. This bloats the image and — dangerously — can bake your **secrets** (`.env`) into a shareable artifact. Always add a `.dockerignore`, and always exclude `.env`.

8.3 docker-compose.yml

Real apps are more than one container — at minimum, your **app** plus a **database**.

`docker-compose.yml` describes them all and wires them together with one command. Here's a complete setup for FastAPI + PostgreSQL:

YAML

```
services:
  # ---- the FastAPI application ----
  app:
    build: . # build from the Dockerfile here
    ports:
      - "8000:8000" # host:container – visit localhost:8000
    environment:
      # Note the host is "db" – the service name below, not localhost.
      DATABASE_URL: postgresql://booklib:secret@db:5432/booklib
      SECRET_KEY: ${SECRET_KEY} # read from your .env file
    depends_on:
      db:
        condition: service_healthy # wait until the DB is actually ready
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000

  # ---- the PostgreSQL database ----
  db:
    image: postgres:16 # official Postgres image, no Dockerfile needed
    environment:
      POSTGRES_USER: booklib
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: booklib
    volumes:
      - pgdata:/var/lib/postgresql/data # persist data across restarts
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U booklib"]
      interval: 5s
      timeout: 3s
      retries: 5

  # ---- optional: a visual DB admin UI ----
  pgadmin:
    image: dpage/pgadmin4
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@example.com
      PGADMIN_DEFAULT_PASSWORD: admin
    ports:
      - "5050:80"

# Named volume so the database survives `docker compose down`.
volumes:
  pgdata:
```

Run the whole stack with one command:

BASH / TERMINAL

```
# Build images and start all services. --build forces a rebuild.
docker compose up --build
```

Key concepts in that file:

- **Service-name networking.** Compose puts all services on one network where they reach each other *by service name*. That's why the app's `DATABASE_URL` host is `db`, not `localhost` — `db` resolves to the database container.
- **volumes**. Containers are ephemeral; when removed, their internal data is lost. A named **volume** (`pgdata`) stores the database files outside the container so your data persists.
- **depends_on** + **healthcheck**. Ensures the app waits until PostgreSQL is genuinely *ready to accept connections*, not just started — avoiding "connection refused" crashes on startup.

TIP

Tip: `docker compose up` starts everything; `docker compose down` stops and removes the containers (but keeps named volumes, so your data survives); `docker compose down -v` also removes the volumes (wiping the database). Use `-v` deliberately, never by habit.

CONCEPT CHECK

Concept Check: In the compose file, why is the database host `db` instead of `localhost`?
Answer: Each service runs in its own container with its own `localhost`. Compose's network lets containers find each other by service name, so the app reaches the database container at the hostname `db`.

8.4 Running in Production Mode

The development command `uvicorn ... --reload` is wrong for production — `--reload` is slow and single-process. Production wants multiple worker processes to use all CPU cores and handle concurrent load.

Multiple workers

BASH / TERMINAL

```
# Run 4 worker processes (rule of thumb: 2 × CPU cores + 1).
uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
```

Gunicorn + Uvicorn workers (the recommended setup)

The production-grade pattern is **Gunicorn** (a robust process manager) running **Uvicorn worker** classes. Gunicorn handles worker lifecycle, restarts, and graceful shutdowns:

BASH / TERMINAL

```
pip install gunicorn

# -w 4 workers; -k uvicorn worker class; bind to all interfaces on 8000.
gunicorn app.main:app -w 4 -k uvicorn.workers.UvicornWorker --bind 0.0.0.0:8000
```

Running migrations on container startup

Your database schema must exist before the app serves traffic. A common pattern runs Alembic migrations, then starts the server, via a small entrypoint:

BASH / TERMINAL

```
# entrypoint.sh
#!/bin/sh
# Apply any pending migrations, then launch the server.
alembic upgrade head
exec gunicorn app.main:app -w 4 -k uvicorn.workers.UvicornWorker --bind 0.0.0.0:8000
```

COMMON MISTAKES

Common Mistakes: Shipping `--reload` to production. It watches the filesystem for changes (pointless in a container), runs a single process (no concurrency), and uses more memory. `--reload` is strictly a development convenience.

8.5 Environment Variables for Production

The **twelve-factor app** methodology (a widely-followed set of best practices for modern apps) says: **store configuration in the environment, never in code**. You already set this up with `pydantic-settings` in Phase 6 — now it pays off in production.

The variables your deployed app needs:

TEXT

```
DATABASE_URL      # the production database connection string
SECRET_KEY         # a long random string for signing JWTs
ALLOWED_ORIGINS   # the front-end domains allowed by CORS
```

These are set on the hosting platform (or in compose), **never** hardcoded or committed. The same image runs in dev, staging, and production — only the environment variables differ. That's the whole idea: one artifact, many environments.

WARNING

Warning: Generate a strong, unique `SECRET_KEY` for production — never reuse the development one, and never commit it. A leaked secret lets anyone forge valid JWTs and impersonate any user. Generate one with `python -c "import secrets; print(secrets.token_urlsafe(32))"`.

8.6 Deployment Platforms Overview

You don't need to manage your own servers. Several platforms take a Dockerised app (or even just your repo) and run it for you, often with a free tier:

Platform	Best for	Notes
Render.com	Beginners	Easiest; free tier; connects to your GitHub repo; managed PostgreSQL
Railway.app	Quick projects	Simple, generous developer experience
Fly.io	Global edge	Runs containers close to users worldwide
AWS ECS / EC2	Large scale	Most powerful and complex; industry standard at scale

Step-by-step: deploy free on Render

1. Push your project (with `Dockerfile`) to a **GitHub repository**.
2. On render.com, create a **New Web Service** and connect that repo.
3. Render detects your `Dockerfile` and builds the image automatically.
4. Add a free **PostgreSQL** instance from Render's dashboard; copy its connection string.
5. In your service's **Environment** settings, set `DATABASE_URL` (the Render Postgres string) and `SECRET_KEY`.
6. Deploy. Render gives you a public URL like `https://your-app.onrender.com`.
7. Visit `https://your-app.onrender.com/docs` — your live, interactive API, on the internet.

TIP

Tip: Free tiers often "sleep" after inactivity, so the first request after idle can take a few seconds to wake the service. That's normal and fine for portfolios, demos, and learning. Paid tiers stay always-on for real traffic.

8.7 Reverse Proxy with Nginx (Concept)

In many production setups, a **reverse proxy** like **Nginx** sits *in front of* your app. A reverse proxy receives all incoming requests and forwards them to your app behind the scenes. Why bother?

- **HTTPS/SSL termination** — Nginx handles encryption certificates, so your app speaks plain HTTP internally while clients get secure HTTPS.
- **Serving static files** efficiently, taking that load off your app.
- **Load balancing** across multiple app instances.
- **Buffering and rate limiting** to protect your app from slow or abusive clients.

A minimal Nginx config that forwards traffic to your FastAPI app:

TEXT

```
server {  
    listen 80;  
    server_name myapi.com;  
  
    location / {  
        proxy_pass http://app:8000;           # forward to the app container  
        proxy_set_header Host $host;  
        proxy_set_header X-Forwarded-For $remote_addr;  
    }  
}
```

TIP

Tip: Managed platforms like Render and Fly.io handle HTTPS and proxying *for you*, so you don't need to configure Nginx yourself when starting out. Understanding the concept matters more than the config at this stage — you'll meet Nginx directly when you manage your own servers.

HANDS-ON

Hands-On Exercise 8: Dockerise and deploy the Book Library

Containerise the Book Library and run it with PostgreSQL, then deploy it live.

1. Create `requirements.txt` :

TEXT

```
fastapi[standard]
uvicorn[standard]
sqlalchemy
alembic
psycopg2-binary
passlib[bcrypt]
python-jose[cryptography]
pydantic-settings
gunicorn
```

2. Create the `Dockerfile` :

DOCKERFILE

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["gunicorn", "app.main:app", "-w", "4", \
    "-k", "uvicorn.workers.UvicornWorker", "--bind", "0.0.0.0:8000"]
```

3. Create `.dockerignore` :

TEXT

```
venv/
__pycache__/
*.pyc
.env
.git
*.db
```

4. Create `docker-compose.yml` (app + PostgreSQL), as shown in 8.3.

5. Build and run locally:

BASH / TERMINAL

```
docker compose up --build
```

6. Verify:

TEXT

```
Visit http://localhost:8000/docs → your interactive API, now in a container
Create a user and a book via /docs
docker compose down → stop everything
docker compose up → restart; your data is STILL there (volume)
```

That persistence-after-restart proves you're using PostgreSQL with a volume, not throwaway in-memory data.

7. Deploy to Render following 8.6, and add the live <https://...onrender.com/docs> URL to your project's README.

What just happened? You packaged your entire application — Python, dependencies, and code — into a portable **image** via the `Dockerfile`, and used **docker-compose** to run it alongside a real **PostgreSQL** container, wired together by service-name networking with a **volume** for persistence and a **healthcheck** so the app waits for the database. You switched from the `--reload` dev server to a production **Gunicorn + Uvicorn-worker** setup, kept all secrets in **environment variables**, and pushed it to a platform that gave you a public HTTPS URL. Your API is now genuinely *deployed* — reachable by anyone, running the same way it ran on your machine. That's the finish line of the core tutorial.

COMMON MISTAKES

Common Mistakes: Hardcoding `localhost` as the database host inside the container, or baking secrets into the image. In containers the database is reached by its **service name** (`db`), and secrets come from **environment variables** at runtime — never from the source or the image.

TIP

Tip: Test your container locally with `docker compose up` *before* deploying. If it runs cleanly in Docker on your machine, it will almost certainly run on the platform — because that's the entire promise of containers: the environment travels with the app.

Checkpoint: At This Point You Can Now...

- ☐ Explain the "**works on my machine**" problem and how containers solve it; distinguish **container vs VM**.
- ☐ Define the four Docker concepts: **image**, **container**, **Dockerfile**, **docker-compose**.
- ☐ Write a **Dockerfile** (with layer-caching order and `--host 0.0.0.0`) and a `.dockerignore`.
- ☐ Write a `docker-compose.yml` running app + PostgreSQL with **service-name networking**, **volumes**, and a **healthcheck**.
- ☐

Run in **production mode** with **Gunicorn + Uvicorn workers** and migrate on startup.



Manage **environment variables** for config/secrets (twelve-factor) and generate a strong `SECRET_KEY`.



Choose a **deployment platform** and deploy free on **Render**.



Explain what an **Nginx reverse proxy** does and why managed platforms often handle it for you.

Phase 8 Summary

PHASE 8 • KEY TAKEAWAYS

In this phase you shipped your API to the world with Docker.

- Docker solves "**works on my machine**" by packaging the app *with* its environment into a portable **container** — lighter and faster than a VM (shipping container vs moving house).
- The four concepts: **image** (template), **container** (running instance), **Dockerfile** (build steps), **docker-compose** (multi-container orchestration).
- A good **Dockerfile** copies `requirements.txt` before the code (layer caching) and binds to `0.0.0.0`; a `.dockerignore` keeps junk and secrets out. **Multi-stage builds** shrink images.
- `docker-compose.yml` runs app + PostgreSQL together: services reach each other by **name** (`db`), a **volume** persists data, and a **healthcheck** makes the app wait for the DB.
- Production uses **Gunicorn + Uvicorn workers** (never `--reload`), runs **migrations on startup**, and reads all config from **environment variables** (twelve-factor); generate a strong `SECRET_KEY`.
- Deploy free on **Render** (or Railway/Fly.io/AWS); a **reverse proxy** (Nginx) handles HTTPS, static files, and load balancing — often managed for you.
- **You containerised the Book Library** with a Dockerfile and compose, ran it on PostgreSQL with persistence, and deployed it to a live public URL.

You've completed the core tutorial — you can build, secure, test, and deploy a real FastAPI application. Now it's time to prove it: the Capstone.

End of Phase 8. Reply "**continue**" for the **Capstone Project — TaskFlow API**, where you'll build a complete production-quality API from specifications using everything you've learned, plus the appendices (cheat sheet, glossary, troubleshooting).